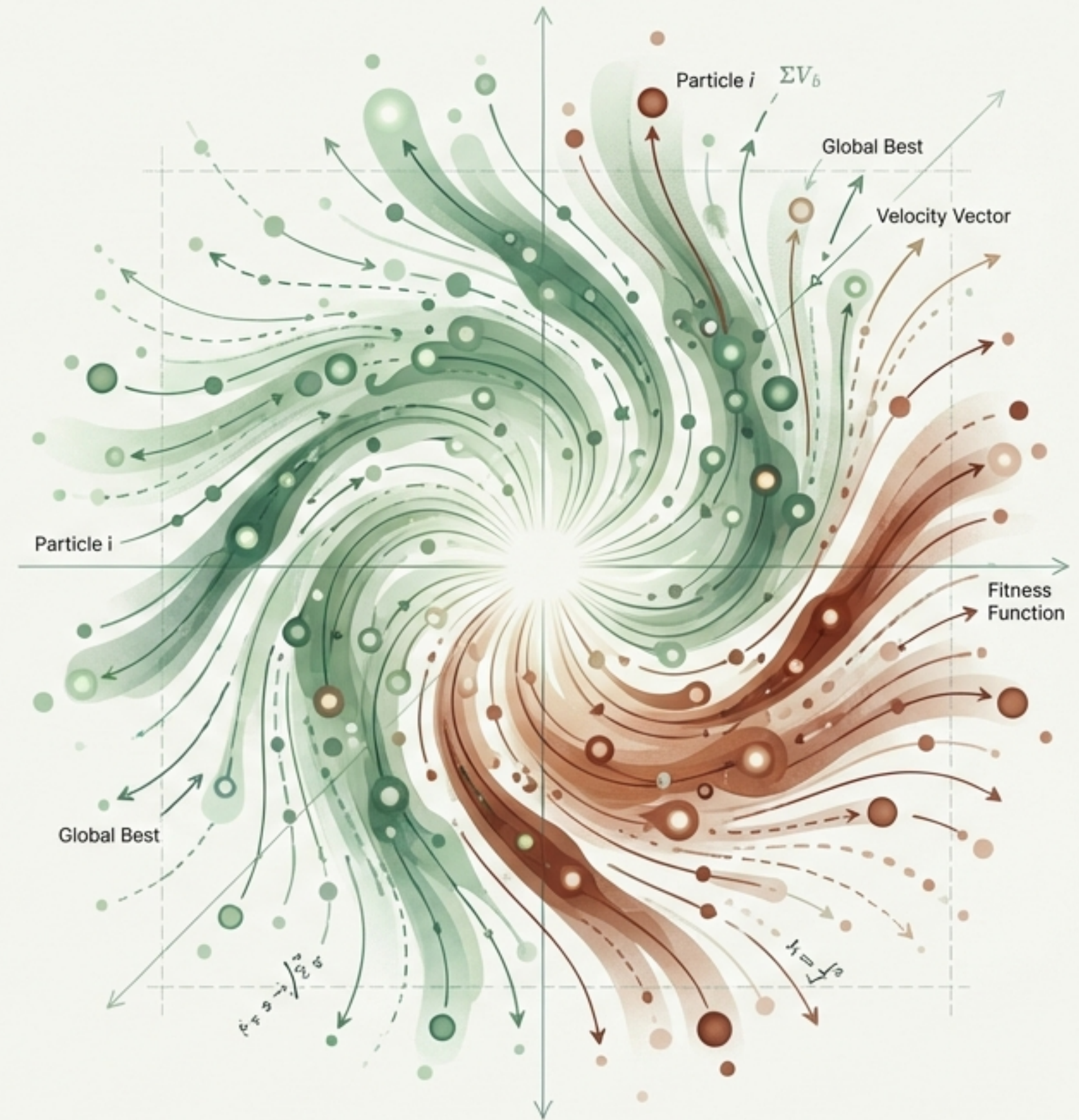




Building & Optimizing Local GenAI Systems

From Prototype Implementation to Evolutionary Optimization

Dr. Kaneeka Vidanage (M.Phil., PhD.)



The Challenge: Secure & Consistent Customer Support

Problem Statement

Customer support teams are overwhelmed by repetitive queries. Standard cloud-based AI solutions raise privacy concerns when handling sensitive customer data.

Key Solution Criteria



Local Execution: The AI must run offline to ensure completely air-gapped data privacy.



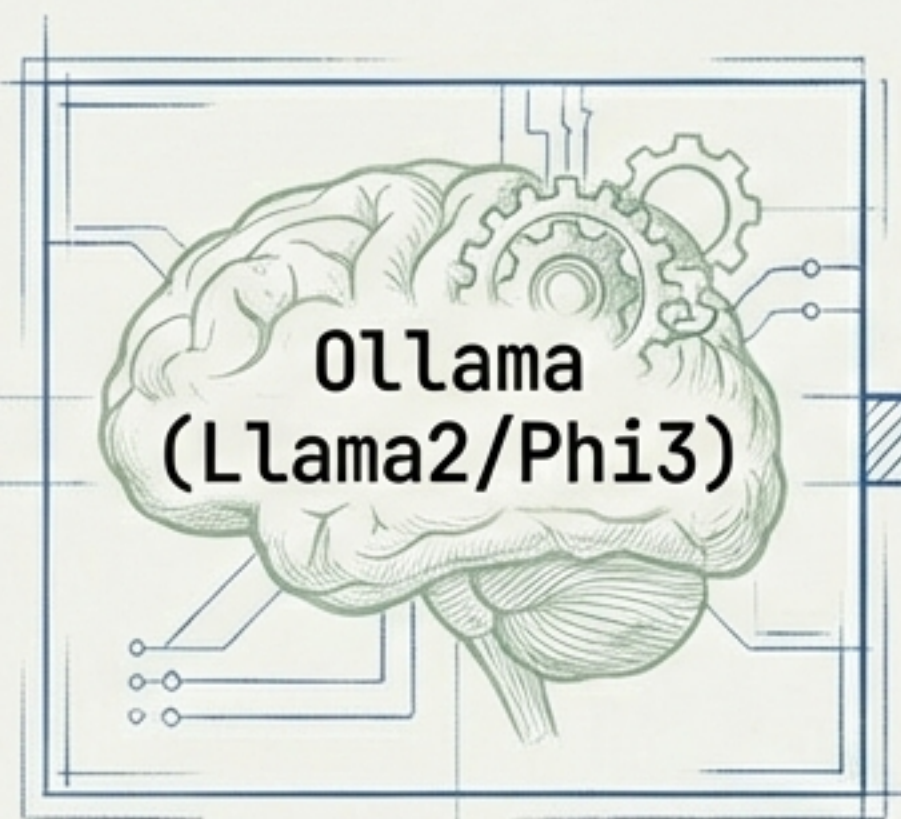
Consistency: Responses must adhere to a strict 'Brand Voice'—polite, empathetic, and professional.



Efficiency: Instant analysis of customer intent and sentiment.

The Architectural Blueprint

The Brain



Local Inference Engine

The Logic



Orchestrates Data Flow

The Face



Agent Interface



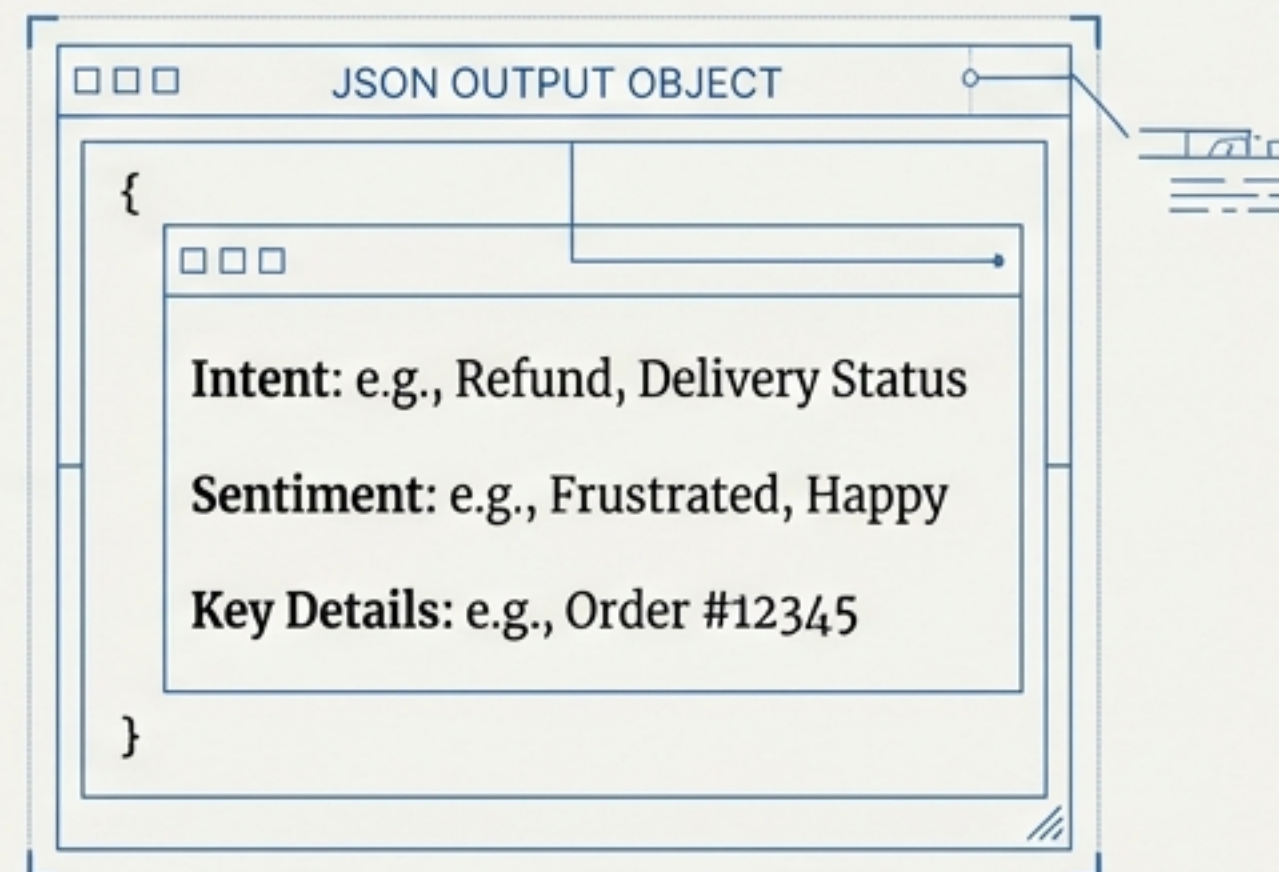
Key Insight: This stack allows for a complete end-to-end application that runs entirely on a standard laptop without internet dependency.

Step 1: The Analyst (Understanding Intent)

```
def analyze_customer_email(email: str) -> str:
    """
    Step 1: Analyze and extract key info (intent, order number, sentiment, etc.)
    """
    extract_prompt = f"""
    Analyze the following customer email.
    1. Classify the intent (refund request, delivery status, product inquiry, compl
    2. Extract key details such as order number, product, sentiment, urgency.
    3. Return the result as JSON with keys: intent, order_number, product, sentimen
    """

    # ... (rest of the function)
```

Structured Analysis



The system uses Chain-of-Thought prompting to first categorize the email before attempting to write a reply.

Step 2: The Writer (Drafting the Response)

```
def draft_reply(email: str, analysis: str) -> str:
    """
    Step 2: Generate support reply with empathy,
    professionalism, and business alignment.
    """
    business_context = """
    You are a professional customer support
    representative at a consumer electronics company.

    Always:
    - Be polite, empathetic, and concise.
    - Acknowledge the customer situation.
    - Include relevant details (order info, delivery
    timing, etc.) when available.
    - End positively, reassuring the customer that we
    value them.
    """
    ...
```

****Context
Injection****

The Synthesis:

1. Input: Customer Email
2. + Analysis Data (from Step 1)
3. + Brand Rules (Polite, Empathetic, Concise)
4. = Final Draft



Rule: "End positively, reassuring the customer that we value them."

Bringing it to Life: The Streamlit Interface

Customer Email

I ordered a Bluetooth speaker last week (Order #12345)... needed it for a birthday gift.

🔍 Analyzing email... Intent: Delivery Status | Sentiment: Anxious

AI Drafted Reply

Subject: Update on Your Order #12345 for Bluetooth Speaker

Dear [Customer Name],

Thank you for reaching out about your order for the Bluetooth speaker. We understand you need it for an upcoming birthday gift.

I've checked the status of Order #12345. [Insert specific delivery details here, e.g., It is currently in transit and estimated to arrive by [Date].]

We appreciate your patience and hope the speaker makes a great gift. Please let us know if you have any other questions.

Best regards,
The Support Team

Regenerate Reply Approve & Send

```
st.title("AI-Powered Customer Support Assistant")
st.write("Paste a customer email and let AI analyze and draft a polite, professional response.")

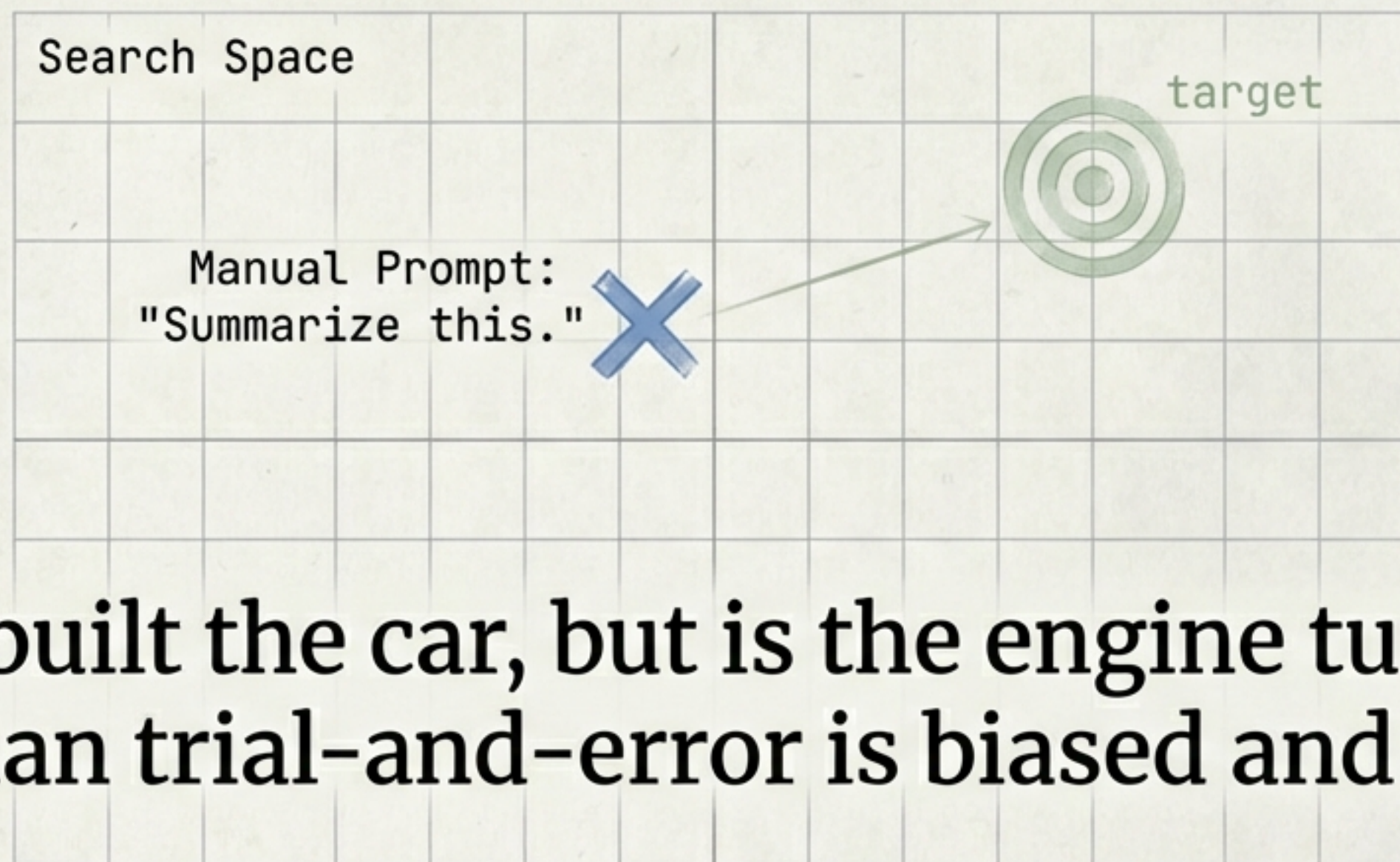
# Input from user
customer_email = st.text_area("Customer Email:", height=200,
                               placeholder="Paste the customer's email here...")

if st.button("Generate Reply"):
    if customer_email.strip():
        with st.spinner("🔍 Analyzing email..."):
            analysis = analyze_customer_email(customer_email)
            st.subheader("Extracted Understanding")
            st.json(analysis)

            with st.spinner("📝 Drafting reply..."):
                reply = draft_reply(customer_email, analysis)
                st.subheader("AI Drafted Reply")
                st.write(reply)
```

With less than 50 lines of Python, we turn backend logic into a usable product for frontline agents.

The Optimization Challenge



We built the car, but is the engine tuned?
Human trial-and-error is biased and slow.

The Solution: Treat prompt engineering as a **mathematical** search problem using **Particle Swarm Optimization** (PSO).

What is Particle Swarm Optimization (PSO)?



A Flock of Birds in Coordinated Flight

Biological Inspiration:

In nature, a flock searches for food. If one bird finds a crumb, it signals the others. The flock adjusts its path based on:

1. **Personal Experience:** Where the individual bird has found food before.
2. **Group Experience:** Where the best food has been found by the neighbor.

The Algorithm:

We use this 'swarm intelligence' to navigate the infinite number of ways to write a prompt.

Translating Nature to Code

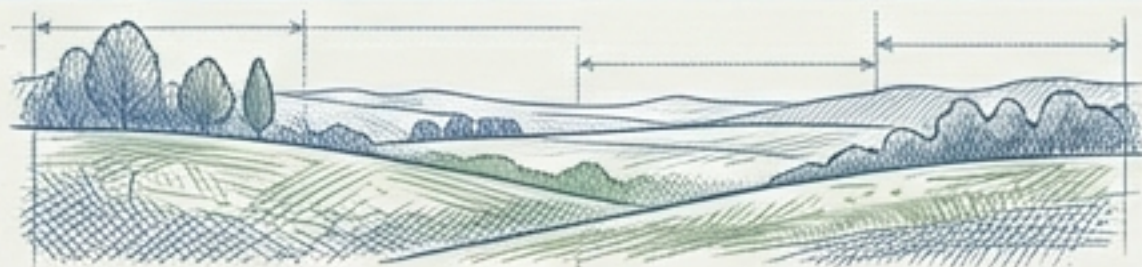
Nature (Biology)

GenAI (Code)



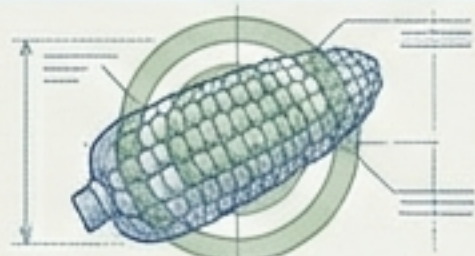
A single bird icon.

The **Particle** = A specific prompt variation (e.g., 'Please summarize').



A wide field/landscape icon.

The **Search Space** = All possible combinations of words.



Food/Corn kernel icon.

The **Food (Target)** = The highest quality response (Fitness Score).



Flight path lines.

Velocity/Mutation = Changing words in the prompt.

The Blueprint of a Particle

```
class Particle:
    def __init__(self, prompt_template: str, dim: int = 1):
        # Each particle's "position" is its current text prompt (string)
        self.position = np.array([prompt_template], dtype=object)

        # how much and in what way the prompt should change. Currently it's defined as zero
        self.velocity = np.zeros(dim, dtype=float) # Placeholder (not really used for text)

        # best version of the prompt so far
        self.best_position = self.position.copy()

        # assign a negative value at the initialization point inside the constructor.
        # hence, even a small improvement becomes noticeable
        self.best_score = -np.inf

        self.velocity = np.zeros(dim, dtype=float)
```

Each particle is an independent object that remembers its own history while exploring the search space.

Current Prompt Text



Personal Best Score

Personal Best Score

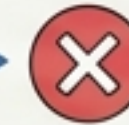
Mutation Factor

Defining Success: The Fitness Function

```
def fitness_function(prompt: str, context_text: str) -> Tuple[float, str]:  
    """Evaluate how good the output is for a given prompt + context."""  
    output = call_ollama(prompt, context_text)  
    #Sometimes things go wrong (network hiccup, invalid prompt, etc.). If the AI's reply starts  
    # with [ERROR: ...], we say: "Okay, that's a flop. Give it the lowest possible score (0.0),  
    #and pass the error back.  
    if output.startswith("[ERROR:"): # error fallback  
        return 0.0, output  
  
    #Count how many words the output has. Why? Longer isn't always better, but a one-word answer  
    #like "Yes" is probably less useful. So this is basically: "More words = at least some effort."  
    length_score = len(output.split())  
  
    #Now check for important keywords inside the AI's response. For example:  
    #if it includes "FAQ," "question," "answer," or "summary." Each time one appears, we count it.  
    # Why? Because those are the words we wanted the AI to emphasize in its answers.  
  
    keyword_hits = sum([kw in output.lower() for kw in ["faq", "question", "answer", "summary"]])  
    # custom score formulae.  
    # Every word adds 0.1 points. (so a 50-word response = 5 points).  
    # Every keyword hit adds 5 points. (so if it used "summary" and "answer" = +10 points).  
    score = length_score * 0.1 + keyword_hits * 5  
    return score, output
```

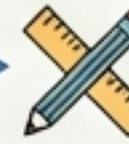
Scorecard

How do we grade the AI?



****Error Check:**

If Error = 0 points.



****Conciseness:**

Length * 0.1 points.

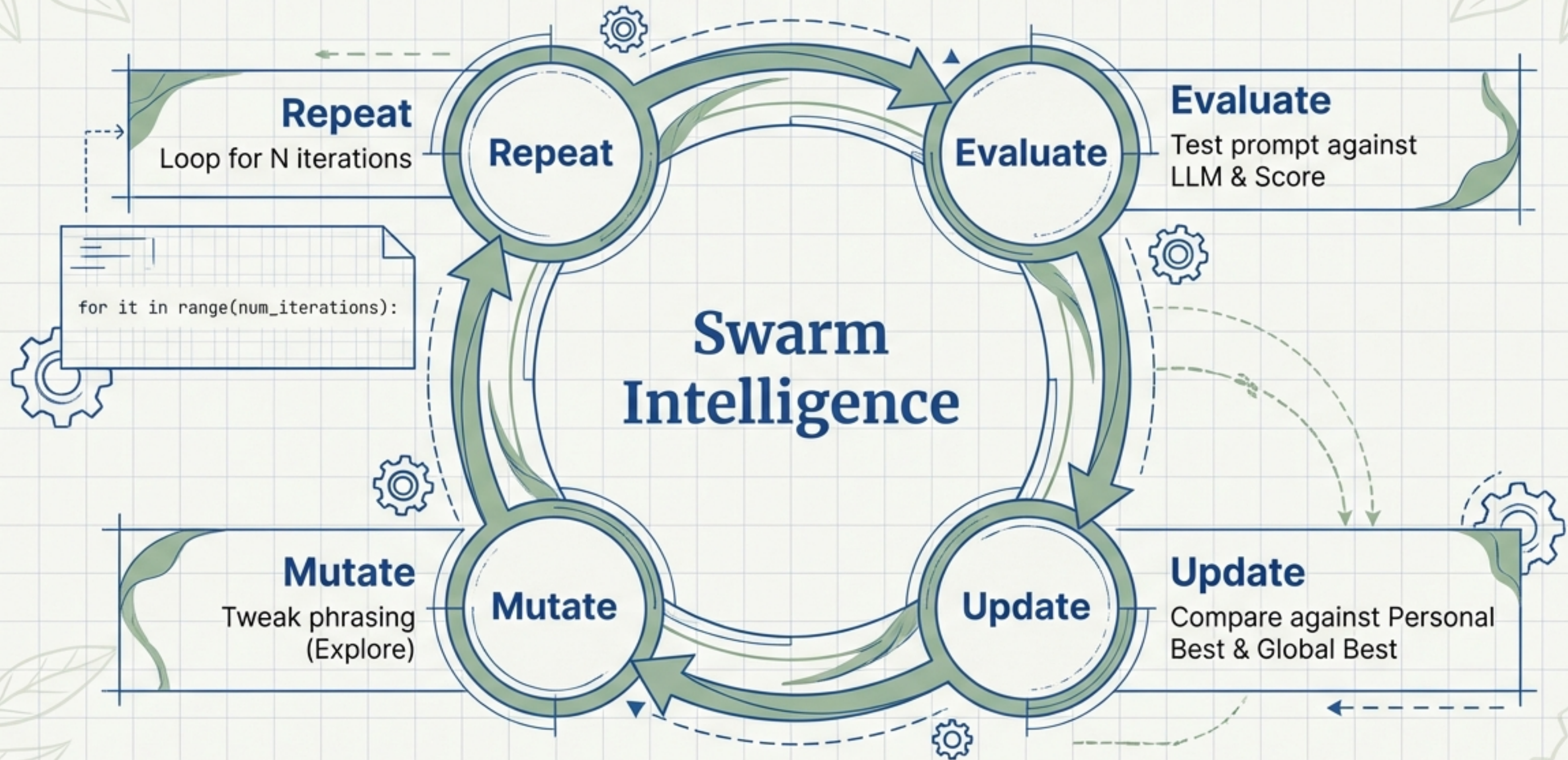


****Keywords:**

Contains 'FAQ', 'Answer', 'Summary'? +5 points per hit.

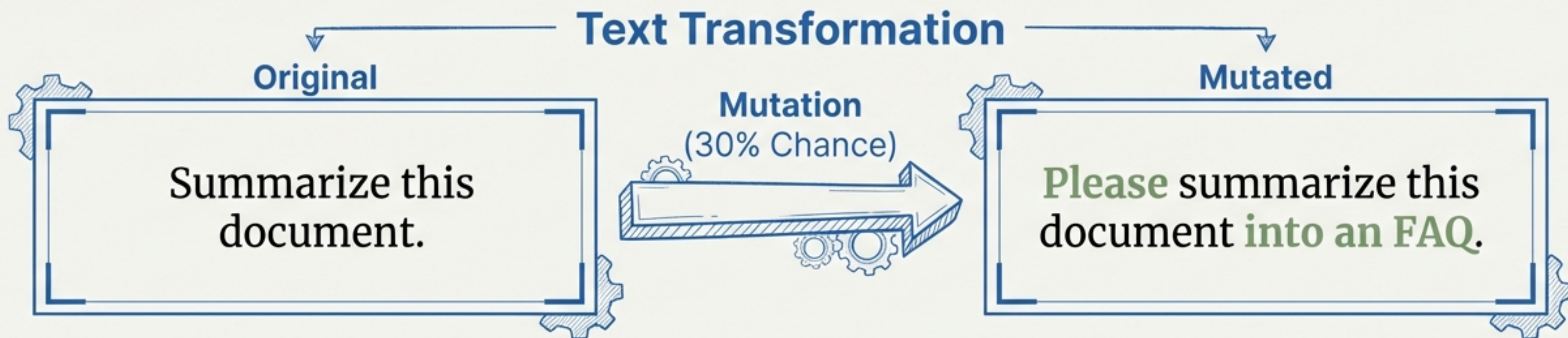
**Total Score = Length
Score + Keyword Bonus**

The Optimization Loop



Mutation: Exploration vs. Exploitation

```
if random.random() < 0.3:
```



The algorithm randomly swaps words or phrases to explore new directions, preventing the swarm from getting stuck in a local optimum.

The Results: Evolution in Action

Console Output

```
=== Iteration 1/2 === Particle 1: Score=0.00 Particle 2: Score=0.00 Particle 3: Score=31.20
```

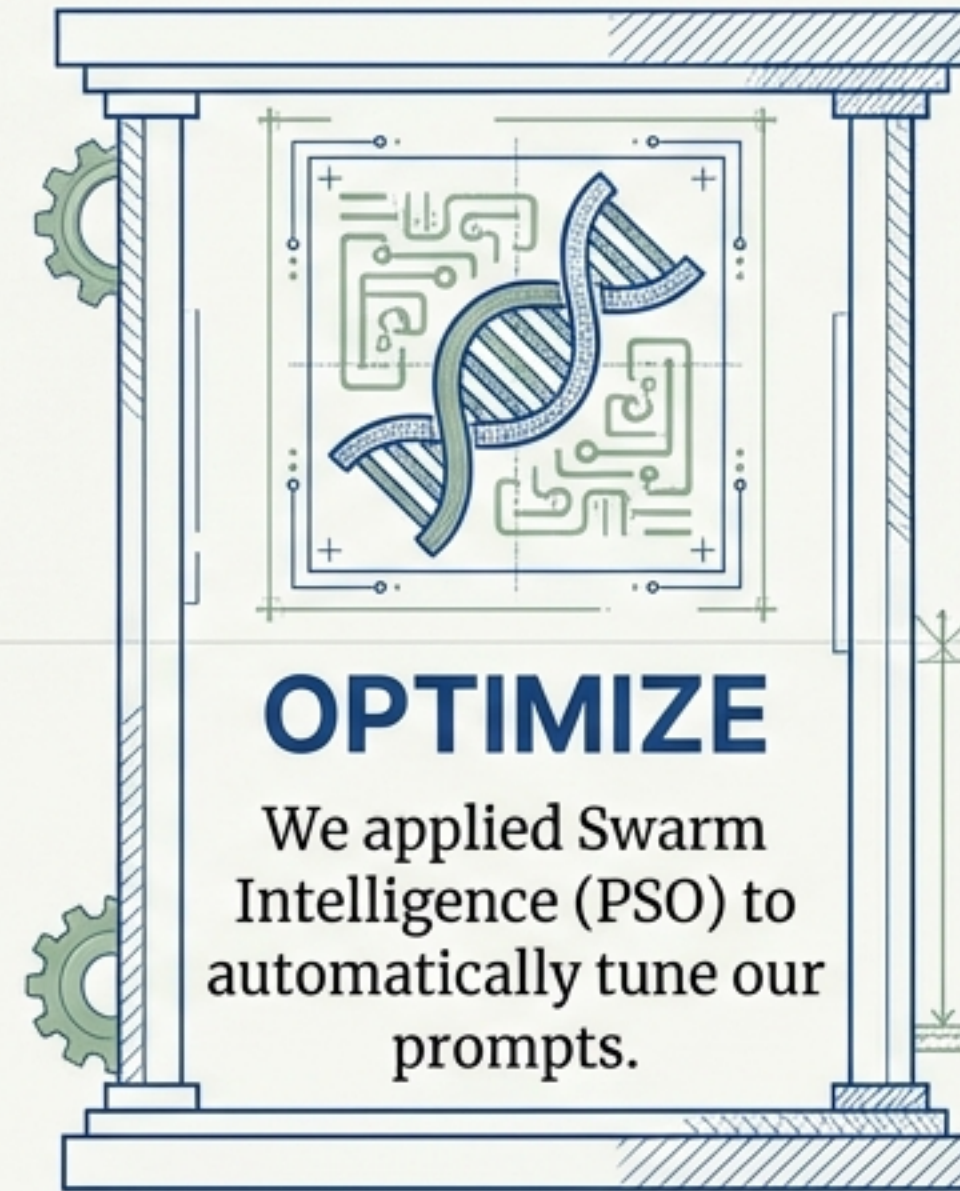
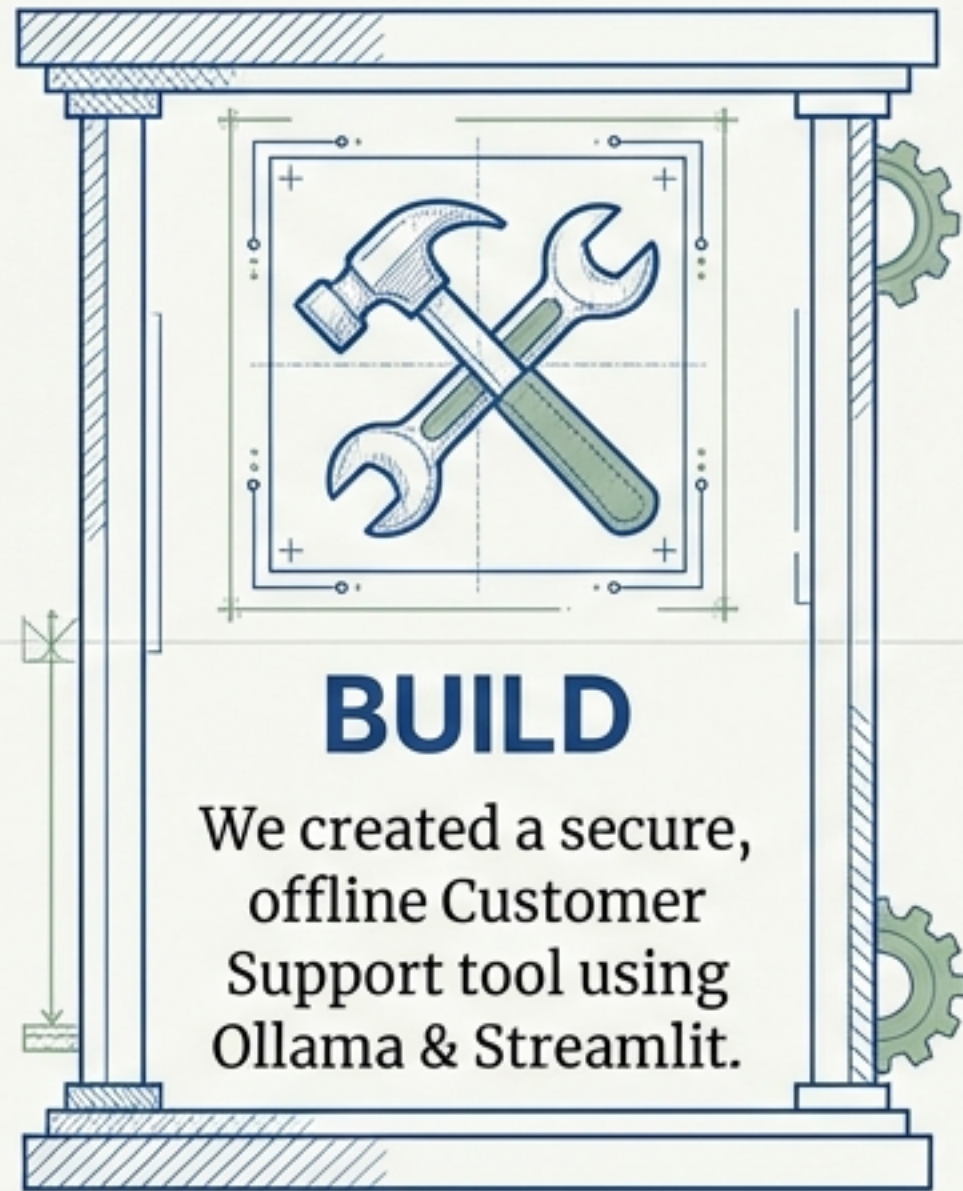
```
=== Iteration 2/2 === Particle 1: Score=30.80
```



****Best Prompt Found:**** "Summarize the following document into an FAQ style with answers:"
****Fitness Score:**** 30.8

The math found a better way to prompt the model than the human did.

Summary & Future Outlook



The future of Generative AI isn't just about using powerful models; it's about using intelligent algorithms to manage and optimize those models automatically.